When Do Language Servers Help Coding Agents?

Ian Barber

July 2026

Abstract

Coding agents are increasingly given language-server tooling on the assumption that better code intelligence makes a better agent. We explored that assumption against a capable baseline of grep, ranged reads and a shell, across 7 models. Making LSP tools available changed nothing: agents largely declined to use the operations they were offered, and every gain required a prompt, training, or a gate that blocked the work. Go-to-definition helped when resolving the receiver's type required a retrieval step rather than being already in context, and only when the agent was instructed to use it. Definition spans saved tokens only when they replaced the file read, and models reread on 35 of 36 pushed spans, and telling them to avoid only removed 2: adjusting the behavior required fine-tuning. On 12 seeded defects, a type checker gating submission took accepted correct outcomes from 1 to 11. Earlier delivery of type errors made no difference, and actively guiding the model to respond to earlier diagnostics did worse than no feedback at all.

> **If you use a coding agent:** run your type checker at the end of the turn as a blocking gate; add navigation tools only as the codebase gets larger, with an instruction to use it, and expect the most gains when resolving a type means reaching into a stub, a generated client or a compiled boundary; expect to save tokens only once the agent stops reading the file, so measure that before counting the saving.

1 Introduction

A language server answers questions about code. An agent with grep, ranged reads and a shell already answers most of them itself, so what a server adds depends on whether it answers a question the agent cannot (IF), whether its answer is cheaper in tokens (FORM), and whether it arrives at a better moment (WHEN). There is significant prior work, but it rarely measures against an agent that already reads code well; that baseline is used throughout.

IF. Typed Holes [2] and LSPRAG [3] push types and definitions into context on the premise the information is missing.

FORM. CodeStruct [4] argues structured reads and edits use fewer tokens than whole-file handling.

WHEN. STALL+ [5] finds static-analysis value varying with integration phase; CoCoGen [1] checks a coherent draft before retrieving context to repair it

2 Method

Most experiments ran in a custom harness: a model drives a fixed loop of six actions over a Python workspace (grep, ranged read, whole-file read, line edit, run tests, submit), and each arm adds or withholds one semantic operation. The definition operation in the retrieval experiments is a static AST resolver rather than a live language server, and we checked that this does not matter: on the 12 synthetic symbols a live Pyrefly daemon answering `textDocument/definition` at the use site agrees with the resolver 12 times out of 12, and on 22 paired real-library cells the two backends produce the same outcome and byte-identical token counts on every cell. Live Pyrefly also served the dispatch lookup arms directly. A case study ran the off-the-shelf shell agent mini-swe-agent with Claude Sonnet 4.5 on 3 SWE-bench tasks (sympy, astropy, sphinx) with an AST-backed definition command, one seed and a 60-step cap. Nothing was tested through an MCP server, skill or editor plugin, so these experiments compare operations rather than delivery surfaces.

Suite	N	Source	Temp x seeds	Isolates
Dispatch ladder	15 tasks x 3 rungs	Synthetic; typed receiver, ~10 same-named overrides (one buggy); type moves from call-site annotation to construction site to factory indirection	0 x 1	Whether lookup reaches targets text search cannot
Hidden rung + control	15 tasks, 450 rollouts	Synthetic; application and test source left on disk but not pasted into the prompt, receiver construction redacted in the quoted asserts	0.7 x 5	Lookup when the type costs a retrieval step
Whole-file contrast	Synthetic set	Fully synthetic	0.7 x 4	Span cost vs a whole-file read
Retrieval cost, paired	11 tasks	Constructed misuse over vendored toolz 1.1.0, more-itertools 11.1.0	0 x 1	Span cost vs grep and ranged reads
Definition spans	12 instances per arm	Mechanically validated	0 x 1	Whether the span displaces the read
Delivery timing	14 tasks x 6 arms, 168 rollouts per arm	Synthetic single-file authoring; no feedback, 2 batched and 3 live delivery points	0.7 x 12	When a diagnostic should arrive
Checker grid	12 defect/clean pairs	Revision task: model reviews a pre-written draft carrying a seeded defect that passes the visible test; clean controls are validated gold	0 x 1	Timing of one diagnostic

Local models were Qwen2.5-Coder-7B, Qwen3.5-27B and Qwen3.6-27B; Claude Sonnet 4.5, DeepSeek v3.1, GLM-4.6 and GPT-5.6 "Luna" ran through an API. Temperature 0.7 with repeated seeds also covered the election and execution-feedback runs and the training harvest; the held-out retest ran at temperature 0. Tasks were constructed because a bounded scan of real repositories produced no candidate passing every screen (appendix). Workspaces are small enough to read any file under budget, so results apply where reading is cheap; static tooling reaches only what a type checker sees, which leaves out dynamic behaviour, wrong annotations and `Any`-heavy boundaries. Code and artifacts: https://github.com/ianbarber/lsps-for-llms.

> **Resolution**: fixed the intended target and passed a held-out test written against the specification, separate from the visible one. **Cost**: total tokens, input plus output, over the whole trajectory. **Election**: how often the agent chose an offered semantic operation. **Substitution**: reads of the defining file after a span was delivered. Checker experiments also report wall time. The unit is the task; intervals are task-level bootstraps.

3 Results

3.1 IF: does delivering the semantic information change what the agent finds?

The first three rungs paste the application and test source into the prompt, so the type is in the agent's context from the first token and the rungs vary only where within that source it sits. There, lookup made no difference. Text search resolved all 15 dispatch tasks; go-to-definition resolved 14 unprompted and 15 prompted, and stayed at 14 or 15 across the 3 rungs. That includes factory indirection, where only a type-aware server can resolve the receiver statically. The agent grepped about 0.3 times per task, read the type wherever it appeared, and opened the buggy override first. The type being present and correct in the code is what carries resolution; the server only queries it.

On matched successes, cost was flat too: 1.03 times text search when lookup was merely available, 0.96 when prompted, 0.94 to 1.06 across rungs. The difference is displacement. The prompted agent swapped the span for its file read (0.07

whole-file reads per task against 1.00); the unprompted agent paid for both (1.00 reads plus 0.80 lookups). Part of that cost is advertisement rather than retrieval: describing the tool adds a one-time 45 tokens to the available arm and 95 to the prompted, and net of that the ratios are 1.00 and 0.90.

When resolving the type costs a retrieval step, the answer changes. The fourth rung stops pasting the application and test source into the prompt. Both stay on disk and either arm can fetch them, so the type is no longer free in context but remains reachable by read, grep or lookup alike; the text arm read the application file in all 75 rollouts. Both arms are also given the use-site line and column, without which the lookup could not be called before a read at all. The rung ran at 5 seeds per task against a matched visible control, in three arms: text search alone (grep_base), lookup available but unprompted (defn_avail), and lookup with an instruction to use it (defn_prompt).

Arm	Resolved	Difference vs grep	Greps	Whole-file reads
grep_base	70/75	—	1.52	2.17
defn_avail	71/75	+0.013 [−0.040, +0.067]	1.47	2.04
defn_prompt	75/75	+0.067 [+0.013, +0.133]	0.36	0.68

Per-task resolution over 5 seeds at temperature 0.7; 95% task bootstrap intervals over 15 tasks; greps and reads are per-task means. In the matched visible control every arm resolved all 75 rollouts.

Prompting raised the resolution rate on 4 tasks and lowered it on none; availability raised it on 3 and lowered it on 2, indistinguishable from text search. Tokens barely moved (0.983 prompted, 1.005 available, on tasks both arms solved); the cost of hiding the type showed up in resolution. The rung is constructed, and it keeps out of the prompt source a real agent would usually be given.

A pilot isolated the resolver on repositories byte-identical apart from one stub. Typed lookup picked the intended override from 8 to 15 same-named candidates; the erased variant resolved to the base declaration every override shares; both type-checked cleanly. The precision made no difference at the agent level: all 12 task-condition cells passed, the typed automatic arm cost 1.037 times the textual ([0.988, 1.093], too wide to call equivalence), every automatic result was followed by a read of the target file, and lookup added about 6 seconds per task.

3.2 FORM: does a compact span cost less than reading the file?

A whole-file read cost 3.35x the span's tokens for Qwen3.6-27B, 3.49x for Sonnet 4.5 and 4.12x for DeepSeek v3.1; no attempt failed under either interface. A capable agent rarely reads a whole file, so the harder test gives the text arm grep, ranged reads and a whole-file fallback. On 11 tasks over real library source, Qwen3.5-27B spent 1,602 mean tokens with text retrieval and 1,235 with definitions, a paired ratio of 1.297 [1.093, 1.527]. Definitions were cheaper on 10 of 11 and both arms solved everything. Here the span did replace the read (none of the 11 was followed by a reread). A span is cheaper only when it replaces the read; the agent's policy decides whether it does.

Election moved easily. An untrained 7B invoked the definition operation in 1 of 48 rollouts; relabel training took that to 48 of 48, ended file reads, cut mean input from 3,086 to 687 tokens, and raised resolution from 31 to 48. Prompt framing raised use of the definition command on 2 of 3 shell-agent tasks. In a second pair of runs with the tool enabled in both arms, the untrained 7B never invoked it in 24 rollouts while the trained model always did. Mean input moved from 2,894 to 689 tokens and success from 14 to 24. Live Pyrefly reproduced the saving once the policy elected; no arm held the policy fixed while varying the backend, so this measures the policy.

Substitution resisted pushing and explicit instruction. Pushed unsolicited, the span was followed by a read of the defining file on 35 of 36 instances. Saying the span was complete removed 2 of those and cost the 27B 294 extra tokens per task.

Model	Reread, pushed span	Reread, plus sufficiency instruction
Qwen3.6-27B	11/12	12/12
Sonnet 4.5	12/12	12/12
DeepSeek v3.1	12/12	10/12

Self-election does not fix it. With Qwen3.6-27B, GLM-4.6 and GPT-5.6 "Luna", the pushed reread reproduced at 35 of 36, and where each model requested the span itself it still reread on 9 of 9, 4 of 9 and 7 of 12. GLM-4.6 solved 6 of these 12 tasks in the pushed arm and 5 of 12 elected; rereads were counted independently of success. The shell-agent case study shows the same habit on real repositories: a manual cross-check matched 16 of 18 definition calls to a later

read of the file just resolved (once via 22 ranged reads of the same sympy module), with the remaining two ambiguous. That agent took 0 to 3 whole-file reads in 44 to 60 actions, so the whole-file ratios above do not describe it.

Training moved substitution. A DAgger-style relabel run [6] harvested 39 demonstrations, none requiring a read, and cut the reread from 11 of 12 to none on held-out instances from different seeds and templates; mean input fell from 1,157 to 748 tokens, a 1.59x saving on tasks both arms solved. Held-out pass moved from 11 to 10 of 12 (1 rescue, 2 losses that pass the visible test and fail the specification), indistinguishable from noise at 12 instances and one seed. A pre-registered confirmation on a reserved 12 instances, disjoint in seeds and templates, reproduced it: reread 12 of 12 to 1 of 12 (11 removed, 1 persisting, none induced), mean input 1,223 to 729 tokens (1.72x on tasks both arms solved), held-out pass 12 to 11 of 12.

Every training example's span contained the defect, leaving open whether the model learned to judge sufficiency or simply to stop reading. We built a sufficiency experiment: 12 instances that hoist each override's arithmetic into a module-level helper, so the span is still the complete definition of the binding method but the defect sits outside it. A twin repository differing in exactly one line restores sufficiency; a third arm leaves the repository byte-identical and delivers a second genuine lookup at the helper.

Reads after span	Span insufficient	Span sufficient	Helper also delivered
Untrained	12/12	12/12	11/12
Trained	12/12	2/12	0/12

The trained model went looking every time the span lacked the defect and largely stopped when it did not; the untrained model read in all three situations. Where reading was necessary the trained model read 1.00 times on average against 3.42; where unnecessary, it read 0.17 times with a sufficient span and 0.00 with the helper also delivered, against 1.58 and 1.75 untrained. Every instance was repaired in every arm on both tests, so the manipulation moved retrieval without moving outcomes, and input tokens were lower in every arm. What training bought is an agent that retrieves only when retrieval is warranted, though that selectivity was tested against a single form of insufficiency. Apparatus checks for the sufficiency experiment, including the adversary search ruling out guessing, are in the appendix.

3.3 WHEN: does the moment the diagnostic arrives change the outcome?

The model is handed an existing draft that passes its visible test but fails a held-out one, and asked to review it and submit. We compared a diagnostic after every edit, one at revision, and a submission gate that rejects a defective draft and asks for repair, against a no-checker control. Accepted submissions both type-clean and correct on the held-out test rose from 1 of 12 to 10 of 12 at revision and 11 of 12 behind the gate (+0.375 [+0.250, +0.500]; +0.417 [+0.292, +0.500]). After-every-edit stayed at 1 of 12 (+0.000 [−0.125, +0.125]). Left to itself the model rarely touched the draft, editing before submitting on 1 of 12 defects, so the checker fired once and 11 of 12 defective drafts went through unchanged. That arm is unexercised rather than refuted in this grid: a diagnostic keyed to edits cannot fire when the agent makes none. A separate experiment, below, tested delivery timing directly.

12 seeded defect/clean pairs	Control	After every edit	One-shot	Gate
Bad completion accepted	11/12	11/12	2/12	1/12
Revision tokens, defect	787	754	1,210	1,380
Wall time, defect (s)	27.8	14.5	68.3	62.7
Revision tokens, clean	591	—	619	591
Wall time, clean (s)	9.4	9.9	9.6	9.5

Mean edits 0.08 against 0.17 in control.

Cost tracks how often the checker fires. Clean drafts needed no diagnostic; there the after-every-edit arm cost 651 tokens, and all 12 clean drafts were accepted everywhere. The zero false rejections are an apparatus check, not a rate: each clean control is its defect's validated gold and the checker deterministic, so it could not have rejected them, and the rate on real work is unmeasured. With today's checkers, the end of the turn is the right place to run one, and the

gate is the better of the two late points because it charges only defective work. It matched control on clean work and spent about 2.2 times control's wall time on defective work: 10 of 12 submissions rejected, each running repair, retest and resubmit, the other 2 repaired before submission.

Its one-task advantage over revision came from refusal: the model received the diagnostic, made no edits, and submitted anyway; only the rejection forced a repair. Its single miss is a limit of the checker itself: a self-repair that was type-clean still failed the held-out test, which no gate can catch.

On a task where the agent does write code, timing itself turns out not to matter. An earlier authoring suite put a 7B agent and a real type checker through six delivery points at 168 paired rollouts each: no feedback, batched at yield, batched after each edit, and three live channels that interrupt mid-generation. Five of the six are indistinguishable, spanning 0.458 to 0.530 resolved with no significant contrast between any pair (smallest p=0.119). The exception is a live channel carrying an instruction to fix each diagnostic on arrival, at 0.339, worse than every other arm including no feedback at all (+0.143 [+0.077, +0.208] for the no-feedback comparison, p=0.0043). Removing that instruction recovers most of the deficit (+0.119 [+0.036, +0.202]). Gating the same channel so it speaks only when the file parses removes about 70% of its messages, since most described the model's own half-finished edit, but barely moves the outcome (+0.024 [−0.065, +0.137]). A 7B on single-file synthetic tasks is a different setting from the 27B revision grid above, so the two do not merge.

Defects had to be seeded because capable models rarely leave checker-detectable ones: frontier models passed every attempt, no natural 7B draft was coherent enough, and coherent 14B drafts were already type-clean. A separate execution-feedback grid (2 frontier models, 14 tasks, 3 seeds, 3 delivery modes) passed all 252 attempts.

4 If you use a coding agent

Run your type checker at the end of the agent's turn and make it blocking. The clearest return here and the cheapest to wire; it costs nothing on clean work. Count how often it blocks, how often the repair passes, and how often it rejects clean work.

Do not wire a checker to nag during the work. Timing itself bought nothing across six delivery points, but a live channel telling the agent to fix each diagnostic as it arrived did worse than no checker at all, mostly by interrupting with complaints about the agent's own half-written code. If you already stream diagnostics, drop any instruction to act on each one, and count how many fire against a file that does not yet parse.

Decide from where your types live, not from what the agent reads. Where a receiver's type is written beside the code that uses it, in an annotated signature or a concrete construction, a navigation tool adds nothing: the model reads the annotation and finds the target itself. It earns its place where knowing the type means leaving that source, as with generated stubs, compiled extensions, vendored packages, or objects assembled by a factory, registry or container. Sample a dozen call sites where a method has several implementations and count how many you can resolve without opening a stub or a generated file; that fraction is what a lookup tool competes against, and you can measure it without running an agent.

If you add one, tell the agent to use it. The merely-available arm performed like text search, so put the instruction in your project instructions or tool description, and count invocations to confirm the agent elects it.

Do not assume a definition tool saves tokens until the agent stops reading. Count reads of the defining file after the tool has returned it; above zero, the tool adds context on top of the read it should replace.

Fine-tuning works, but only if you own the weights, which rules it out for a hosted agent. It buys a conditional policy: the trained model still went looking whenever what it held was insufficient. Verify by counting post-span reads on held-out tasks before and after training.

Keep annotations correct and present; the text baseline exploited that. Check by counting wrong-file edits on your workload.

5 Future work

Retrieval calibration. Training produced a model that retrieves when what it holds is inadequate and stops when it is not, behaviour consistent with a learned sufficiency judgment, acquired from demonstrations that never exercised it, and nothing about it is specific to definition spans. Agents retrieve constantly and mostly indiscriminately: grep hits, file chunks, documentation, search results, other tools' output. If "do I have enough?" is trainable, it is a general lever on agent cost. Does it transfer across retrieval channels, survive forms of inadequacy unlike the delegation tested here, and hold at other scales, outside a task-specific adapter, on codebases where sufficiency is a matter of degree?

Tools co-designed with the agent. Every tool tested was built for a human, who can ignore a diagnostic and choose what to ask. An agent inherits those defaults, and the results suggest the inherited default that costs most is not the timing but the demand: interrupting mid-work was harmless once the agent was no longer told to act on each message, while gating the interruptions removed most of them and changed little. So the question is less when a checker should speak than what it should ask for, and whether it can learn to distinguish a diagnostic worth acting on now from one that will resolve itself. A server could likewise anticipate what the caller needs next rather than answering only what it was asked. Neither is tested here.

Assessing a codebase in advance. The discriminator, whether resolving a receiver's type costs a retrieval step, is a repository property no practitioner can yet compute, and the sampling test above is only a proxy for it. Annotation density, indirection depth between call site and binding definition, or the share of types resolving only through a stub or compiled boundary might predict what a server buys.

Which tasks. Everything here is dispatch ambiguity and small seeded defects in readable workspaces. Semantic tooling may matter in proportion to how much relevant context sits outside what the agent can afford to read: wide refactors, cross-module renames, API migrations. An agent that stops reading is cheaper until the thing it needed was in the part it skipped; we do not know which tasks that failure lands on.

6 Conclusion

A language server helped where it changed the agent's behaviour, a pattern shared by all three operations despite different models, suites and measures; each needed its own mechanism: an instruction for lookup, a blocking gate for the checker, fine-tuning for substitution. The checker paid off as a gate at the end of the turn, cheap to wire, free on clean work. Go-to-definition mattered only where the type sat outside the source at hand, and then only when prompted. Compact spans cost less than reading the file only if the agent stops reading it, and mostly it did not. Fine-tuning removed that habit on one model in one apparatus, establishing the behaviour is trainable without saying how to obtain it in an agent whose weights you do not control.

7 Appendix: task construction and apparatus checks

Checker defects were screened so that each is coherent (the target file parses and leaves no unimplemented stub), passes the visible test, fails a held-out test, and carries a single semantic diagnostic in the target file; each clean control is that defect's own validated gold. The screens on the repository scan were environment, leakage, ambiguity and discriminating lookup.

In the sufficiency experiment, a scripted adversary searched 17 one-parameter repair forms and found none that reached the held-out test without retrieval, so the instances cannot be solved by guessing. The harness also re-shows any file the agent has edited, which would have delivered the hidden line without a read; we redacted that post-edit view while leaving deliberate reads and grep available.

References

[1] Zhangqian Bi, Yao Wan, Zheng Wang, Hongyu Zhang, Batu Guan, Fangxin Lu, Zili Zhang, Yulei Sui, Hai Jin, and Xuanhua Shi. Iterative refinement of project-level code context for precise code generation with compiler feedback. In *Findings of the Association for Computational Linguistics*, 2024. CoCoGen; arXiv:2403.16792.

[2] Andrew Blinn, Xiang Li, June Hyung Kim, and Cyrus Omar. Statically contextualizing large language models with typed holes. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA2), 2024. arXiv:2409.00921.

[3] Gwihwan Go, Quan Zhang, Chijin Zhou, Zhao Wei, and Yu Jiang. LSPRAG: LSP-guided RAG for language-agnostic real-time unit test generation. *arXiv preprint arXiv:2510.22210*, 2025.

[4] Myeongsoo Kim, Joe Hsu, Dingmin Wang, Shweta Garg, Varun Kumar, and Murali Krishna Ramanathan. CODE-STRUCT: Code agents over structured action spaces. *arXiv preprint arXiv:2604.05407*, 2026.

[5] Junwei Liu, Yixuan Chen, Mingwei Liu, Xin Peng, and Yiling Lou. STALL+: Boosting LLM-based repository-level code completion with static analysis. *arXiv preprint arXiv:2406.10018*, 2024.

[6] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011. PMLR v15:627–635.